# Compositional Skill Routing for LLM Agents: Decompose, Retrieve, and Compose

**Anonymous**

## Abstract

LLM agents increasingly rely on external skills—reusable tool specifications—but real-world tasks often require *composing* multiple skills, not just selecting one. We formalize this as the **Compositional Skill Routing** problem: given a complex user query and a large skill library, decompose the query into atomic sub-tasks, retrieve the appropriate skill for each sub-task, and compose an executable plan. We present SKILLWEAVER, a decompose-retrieve-compose framework combining an LLM task decomposer, a bi-encoder skill retriever with FAISS indexing, and a dependency-aware DAG planner. To support evaluation, we introduce COMPSKILLBENCH, a benchmark of 300 compositional queries over 2,595 skills spanning 17 categories. Our experiments reveal that *task decomposition quality is the primary bottleneck*: oracle decomposition achieves 99.5% skill recall, while standard LLM decomposition reaches only 33.9% at the category level. To address this, we propose *Skill-Aware Decomposition* (SAD), a retrieval-augmented approach that informs the decomposer about available skills, improving category recall to 54.2% (+60%) consistently across five models (7–72B); notably, a 7B model with SAD outperforms a 72B model without it. SKILLWEAVER further reduces context window consumption by over 99% compared to exposing all tools.

## 1 Introduction

The agent paradigm for large language models (LLMs) has evolved beyond single-turn generation to encompass tool use, planning, and multi-step task execution (Schick et al., 2023; Qin et al., 2023; Patil et al., 2024). A key architectural pattern emerging in modern LLM agents is the use of *skills*: modular, reusable tool specifications that define specific capabilities along with instructions for when and how to invoke them (Anthropic, 2025). We use *skill* following Anthropic's SKILL.md specification; skills differ from traditional APIs in their emphasis on structured natural language documentation and composability metadata. As agent skill libraries grow—with repositories already containing thousands of community-contributed skills—a fundamental routing question arises: *given a user query, which skill(s) should the agent invoke?*

Prior work treats skill routing as single-skill selection (Zheng et al., 2025), but real-world queries frequently require *multiple* skills—e.g., "*Download the dataset, transform it, and create visual reports*" needs an API client, a data processor, and a visualization tool.

We formalize this as **Compositional Skill Routing** (Figure 1): given query $q$ and skill library $\mathcal{S}$, produce an ordered sequence of skills $[s_1, \ldots, s_k]$ where each $s_i$ handles one atomic sub-task.

We present SKILLWEAVER, a three-stage framework that addresses this problem through:

1. **Decompose**: An LLM-based task decomposer that breaks complex queries into atomic sub-tasks, each requiring exactly one skill.
2. **Retrieve**: A bi-encoder retriever that identifies candidate skills for each sub-task using semantic similarity over skill metadata.
3. **Compose**: A compatibility-aware planner that selects the best skill per step while considering inter-skill compatibility, producing an executable DAG.

To evaluate compositional skill routing, we construct COMPSKILLBENCH, the first dedicated benchmark for this task. COMPSKILLBENCH contains 300 compositional queries over 2,595 curated skills spanning 17 functional categories, with ground-truth skill chains and three difficulty levels. Skills are sourced from 212 public GitHub repositories and deduplicated to ensure quality.

Our experiments yield several key findings:

- **Decomposition is the bottleneck**: With oracle decomposition, our retriever achieves 99.5% Recall@1 and perfect category accuracy. Standard LLM decomposition drops to 33.9% CatR@1, identifying decomposition as the critical frontier.
- **Skill-aware decomposition closes the gap**: Our proposed *Skill-Aware Decomposition* (SAD), which provides the decomposer with retrieved skill context, improves CatR@1 from 33.9% to 54.2% (+60%) without changing the underlying model.
- **Metadata suffices for retrieval**: Metadata-only encoding matches or outperforms body-aware encoding, suggesting concise skill metadata carries the most discriminative signal.

## 2 Related Work

**Tool Selection and Routing.** API retrieval (Patil et al., 2024; Qin et al., 2023), documentation matching (Hao et al., 2024), and hierarchical routing (Zheng et al., 2025) study single-tool selection. SkillRouter (Zheng et al., 2025), closest to our work, uses a bi-encoder for single-skill routing. Hierarchical/self-reflective agents (Du et al., 2024) and tool-creation frameworks (Yuan et al., 2025) scale tool use, but still treat selection as a single-tool or per-step problem. None jointly optimizes decomposition, retrieval, and inter-skill compatibility for compositional tasks.

**Tool-Augmented LLM Benchmarks.** API-Bank (Li et al., 2023), ToolQA (Zhuang et al., 2024), and TaskBench (Shen et al., 2023) benchmark tool use but over fixed or small tool sets. Our CompSkillBench is the first for compositional *routing* over thousands of skills.

**Task Decomposition and Planning.** Prompting strategies (Wei et al., 2022; Zhou et al., 2022), planning frameworks (Huang et al., 2022; Wang et al., 2023), and agentic systems (Yao et al., 2023) explore LLM decomposition. We contribute *skill-aware* decomposition: the decomposer is informed by the skill library itself via a retrieval-augmented feedback loop.

**MCP Ecosystem and Tool Discovery.** The MCP protocol (Anthropic, 2024) standardizes agent–tool integration with 10,000+ servers. Progressive discovery (Qin et al., 2023) addresses tool overload systemically; we focus on *semantic routing*—which skills to compose given a query.

**Retrieval-Augmented Generation.** We adapt bi-encoder retrieval (Karpukhin et al., 2020) for skills, extending it upstream to inform decomposition via retrieved hints.

## 3 Problem Formulation

**Skill Library.** A skill library $\mathcal{S} = \{s_1, \ldots, s_N\}$ contains $N$ skills. Each skill $s_i$ is a tuple $(n_i, d_i, b_i, C_i)$ where $n_i$ is the name, $d_i$ is a natural language description, $b_i$ is the full specification body (instructions, examples, configuration), and $C_i \subseteq \mathcal{C}$ is a set of functional categories from a taxonomy $\mathcal{C}$.

**Compositional Skill Routing.** Given a complex query $q$ that requires multiple capabilities, the goal is to produce:

1. A decomposition $D(q) = [t_1, \ldots, t_K]$ of $K$ atomic sub-tasks.
2. A skill assignment $\sigma : [t_1, \ldots, t_K] \to \mathcal{S}^K$ mapping each sub-task to a skill.
3. An execution plan (DAG) $G = (V, E)$ specifying dependencies between steps.

The compositional routing function is $f : q \to (D, \sigma, G)$ optimizing:

$$\max_{D,\sigma,G} \alpha \sum_{k=1}^{K} \text{rel}(t_k, \sigma(t_k)) + (1-\alpha) \sum_{(i,j)\in E} \text{compat}(\sigma_i, \sigma_j) \quad (1)$$

where $\text{rel}(\cdot)$ measures sub-task–skill relevance, $\text{compat}(\cdot)$ measures inter-skill compatibility, and $\alpha \in [0, 1]$ controls the relevance–compatibility trade-off (instantiated in Eq. 5). While joint optimization of Eq. 1 is intractable in general, our cascaded pipeline (§4) provides a tractable approximation; SAD (§4.4) further tightens this by feeding retrieval signals back into decomposition.

## 4 Method: SkillWeaver

SkillWeaver implements compositional skill routing through three cascaded stages (Figure 1).

### 4.1 Stage 1: Task Decomposition

Given a complex query $q$, the task decomposer uses an instruction-tuned LLM to produce an ordered list of atomic sub-tasks:

$$D(q) = \text{LLM}(p_{\text{sys}}, p_{\text{user}}(q)) = [t_1, \ldots, t_K] \quad (2)$$

where $p_{\text{sys}}$ instructs the model to output sub-tasks as a JSON array of strings, each requiring exactly one skill.

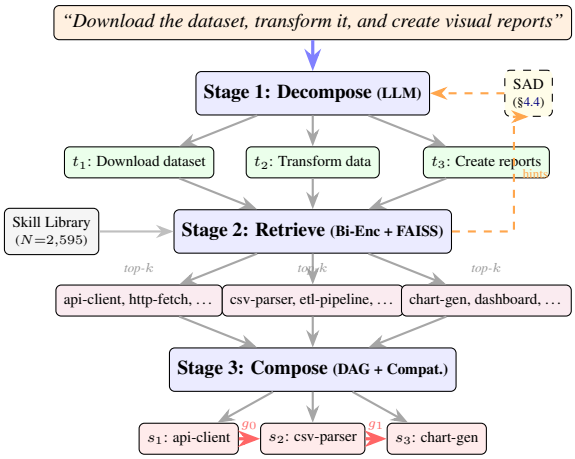


Figure 1: Overview of SKILLWEAVER. A query is decomposed into sub-tasks, each matched to skills via bi-encoder retrieval, then composed into a DAG. Dashed arrows: SAD feedback loop (§4.4).

### 4.2 Stage 2: Skill Retrieval

For each sub-task $t_k$, we retrieve the top-$m$ candidates using a bi-encoder (BGE-large-en-v1.5 (Xiao et al., 2023)):

$$\text{cand}(t_k) = \text{top-}m_{s \in \mathcal{S}} \ \cos(E_q(t_k), E_s(s)) \quad (3)$$

We compare two representations: **metadata-only** ($n_s \oplus d_s$) and **body-aware** ($n_s \oplus d_s \oplus b_s[:2000]$). Embeddings are $L_2$-normalized and indexed with FAISS (Johnson et al., 2019).

### 4.3 Stage 3: Dependency-Aware DAG Planning

Unlike prior work that assumes a fixed sequential execution order, SKILLWEAVER infers task dependencies and identifies parallelizable sub-tasks, producing a *directed acyclic graph* (DAG) as the execution plan.

**Dependency Detection.** Given subtasks $[t_1, \ldots, t_K]$, we detect pairwise dependencies via I/O type overlap, producer–consumer keyword patterns, and linguistic markers (e.g., "then", "after"). If none are detected, the planner falls back to sequential chaining.

**Parallel Group Assignment.** Given dependency edges $E$, we assign each sub-task to a parallel group $g(t_k)$ via topological level assignment (Kahn's algorithm):

$$g(t_k) = \begin{cases} 0 & \text{if indeg}(t_k) = 0 \\ \max_{(t_i, t_k) \in E} \ g(t_i) + 1 & \text{otherwise} \end{cases} \quad (4)$$

Sub-tasks with the same group number have no mutual dependencies and can execute concurrently, reducing the critical-path length from $K$ (fully sequential) to the DAG depth $d \leq K$.

**Multi-Dimensional Compatibility.** We measure compatibility via four signals ($\mathcal{D} = \{\text{io}, \text{cat}, \text{tag}, \text{kw}\}$): I/O type coercion, category/tag Jaccard similarity, and keyword co-occurrence, weighted $(0.4, 0.2, 0.2, 0.2)$.

**DAG-Aware Skill Selection.** The final skill per step combines retrieval relevance with DAG-predecessor compatibility:

$$\sigma(t_k) = \arg \max_{s \in \text{cand}(t_k)} \alpha \cdot \text{sim}(t_k, s) + (1-\alpha) \cdot \bar{c}_k(s) \quad (5)$$

where $\bar{c}_k(s) = |\text{pred}(k)|^{-1} \sum_{i \in \text{pred}(k)} \text{compat}(\sigma_i, s)$ averages compatibility with all DAG predecessors.

### 4.4 Skill-Aware Decomposition (SAD)

A key insight is that LLM decomposers produce generic descriptions poorly aligned with skill metadata. We propose *Skill-Aware Decomposition* (SAD): given initial decomposition $D^{(0)}(q)$, retrieve top candidates for each sub-task, construct a hint set $\mathcal{H}$ of the top-$H$ skill names, and re-prompt the decomposer:

$$D^{(1)}(q) = \text{LLM}(p_{\text{sys}}, p_{\text{SAD}}(q, \mathcal{H})) \quad (6)$$

SAD works even when Pass 1 is poor: imprecise sub-task descriptions still contain keywords that surface relevant skills, providing a *vocabulary bridge* between user queries and skill names (Algorithm 1).

**Algorithm 1** Skill-Aware Decomposition (SAD)

**Require:** Query $q$, skill library $\mathcal{S}$, retriever $R$, hint count $H$
**Ensure:** Refined decomposition $D^{(1)}(q)$

```
Pass 1 (Vanilla): D^(0)(q) = [t_1, ..., t_K] ← LLM(p_sys, q)
for k = 1 to K do
    cand(t_k) ← R.retrieve(t_k, top_k=H)
end for
H ← top-H skills from ⋃_k cand(t_k)
Pass 2 (Skill-Aware): D^(1)(q) ← LLM(p_sys, p_SAD(q, H))
return D^(1)(q)
```

| Category | Count | Examples |
|---|---|---|
| Code Generator | 389 | prd-generator, nextjs-dev |
| API Client | 312 | stripe-api, github-api |
| Test Writer | 248 | unit-test-gen, qa-suite |
| Data Processor | 221 | csv-parser, etl-pipeline |
| Code Analyzer | 187 | code-review, lint-checker |
| Writer | 176 | blog-writer, doc-gen |
| Deployment | 142 | k8s-deploy, ci-cd-setup |
| Security | 118 | vuln-scanner, audit-tool |
| Visualizer | 97 | chart-gen, dashboard-builder |
| Search | 96 | web-search, code-search |
| *+ 7 more categories* | | 609 total |

Table 1: Top 10 skill categories in CompSkillBench (of 17 total). The full pool contains 2,595 skills from 212 repositories.

## 5 Benchmark: CompSkillBench

### 5.1 Skill Pool Construction

We collect skills from 212 public GitHub repositories that adopt the SKILL.md specification format (Anthropic, 2025). The raw collection yields 11,011 skills. We apply the following curation pipeline:

1. **Deduplication**: Skills with identical names are merged, reducing to 7,681 unique skills.
2. **Quality filtering**: Skills without descriptions or with descriptions shorter than 10 characters are removed.
3. **Categorization**: Skills are assigned to 17 functional categories (Table 1) using keyword matching on name and description fields only. Anti-keywords prevent false positive assignments (e.g., "debug" skills are excluded from "deployment"). Skills matching no category or multiple conflicting categories are excluded.
4. The final pool contains **2,595** categorized skills.

### 5.2 Query Generation

Compositional queries are generated by combining skills from different categories into multi-step tasks:

**Difficulty Levels.**
- **Easy** (117 queries): 2 skills, 2 categories
- **Medium** (120 queries): 3 skills, 3 categories
- **Hard** (63 queries): 4–5 skills, 4–5 categories

Each query is associated with ground-truth sub-task descriptions (derived from actual skill descriptions), ground-truth skill IDs, required categories, and a sequential execution order. The benchmark totals 300 queries involving 523 unique ground-truth skills.

**Text Overlap Disclosure.** Ground-truth descriptions are derived from skill metadata, producing high lexical overlap (ROUGE-L=0.845, 37.8% exact copies). This inflates oracle scores; human-written queries (§7.4) show lower absolute performance but consistent *relative* SAD gains (+20.8pp template, +13.8pp human).

### 5.3 Evaluation Metrics

We evaluate at three granularities:

**Step-Level Metrics.**
- **Skill Recall@$k$** (R@$k$): Fraction of steps where the ground-truth skill appears in the top-$k$ candidates.
- **Category Recall@$k$** (CatR@$k$): Fraction of steps where *any skill from the correct category* appears in the top-$k$. This relaxed metric is more practical, as many skills within a category are functionally interchangeable.

**Chain-Level Metrics.**
- **Chain Exact Match**: Fraction of queries where *all* steps select the exact ground-truth skill.
- **Chain Category Match** ($\text{Chain}_{\text{cat}}$): Average fraction of steps per query that select a skill from the correct category.

**Decomposition Accuracy.** Fraction of queries where the predicted number of sub-tasks matches the ground truth. Note that DA is a strict structural metric; a query with 3 ground-truth steps decomposed into 4 (with one additional valid intermediate step) receives DA=0. We use DA primarily to diagnose decomposition granularity; CatR@1 is the primary quality metric.

## 6 Experimental Setup

**LLM Decomposers.** Five open-weight models: Qwen2.5 (7B/14B/72B) (Qwen Team, 2024), Llama-3.1-8B (Dubey et al., 2024), Mistral-7B (Jiang et al., 2023). Generation: $\tau = 0.1$,

$\text{top}_p = 0.9$, max 256 tokens. Low variance confirmed: CatR@1 std $\leq$0.003 across 3 seeds.

**Retriever.** BGE-large-en-v1.5 (Xiao et al., 2023) (1024-dim) serves as the bi-encoder, with FAISS IndexFlatIP for exact inner product search. We set $k = 10$ for retrieval unless otherwise noted.

**Baselines.**

- **Single-Skill**: No decomposition; full query retrieves one skill set.
- **LLM-Direct**: No decomposition; LLM selects from top-50 retrieved skills.
- **Oracle**: Ground-truth sub-tasks for retrieval (upper bound).
- **ReAct**: Iterative skill selection via think-act-observe loop.

**Hardware.** All experiments run on NVIDIA V100-SXM2-16GB GPUs. 7B–14B models use a single V100; the 72B model is served in 4-bit quantization (NF4 via `bitsandbytes`) across 8 V100s with an explicit per-GPU `max_memory` map (no CPU offloading; verified post-hoc by `accelerate` log inspection). Generation uses the same decoding hyperparameters across all models.

# 7 Results

## 7.1 Main Results

Table 2 presents the main experimental results across all configurations.

**Key findings.** Oracle decomposition achieves R@1 = 99.5%, but the best LLM decomposer (Qwen2.5-7B) reaches only CatR@1 = 36.3%, confirming decomposition as the primary bottleneck. Our full pipeline (CatR@1 = 33.9%) substantially outperforms the single-skill baseline (21.1%), validating that even approximate decomposition aids compositional routing. Metadata-only retrieval matches body-aware (oracle R@1: 99.5% vs 99.0%) with lower encoding cost, suggesting concise metadata captures the essential signal.

## 7.2 Decomposer Comparison

Qwen2.5-7B significantly outperforms Mistral (17.7%) and Llama (0%), with decomposition patterns (Figure 2) revealing that Llama systematically over-decomposes (median 9 sub-tasks vs ground-truth peak at 2–3), while Qwen most closely aligns with the true distribution.

## 7.3 Difficulty Analysis

The decomposition-based pipeline shows its *greatest advantage on harder queries*: for hard queries (4–5 skills), Qwen + Meta achieves CatR@1 = 41.0% versus the single-skill baseline at 14.6% (+181%); see Table 7 in Appendix A. This pattern suggests that decomposition becomes increasingly important as task complexity grows.

## 7.4 Skill-Aware Decomposition Results

**SAD ablation.** Table 4 presents the ablation over hint count $H$. DA increases monotonically (36.0% to 79.0%), but CatR@1 peaks at $H$=15 (54.2%) before declining at $H$=25 (47.4%), revealing a noise–alignment trade-off. The disproportionate jump from $H$=0 to $H$=5 (+38% CatR@1) suggests that even a small number of hints fundamentally shifts the decomposer from free-form to skill-aware generation, with diminishing returns from additional hints.

**Cross-model generalization.** SAD generalizes across model families (CatR@1 gains: +26.9% to +59.9%), with **7B+SAD (54.2%) outperforming 72B vanilla (36.8%)**. The 72B model shows *hint-induced consolidation*: SAD reduces its DA from 62.7% to 50.3% (avg. 2.3 vs. 2.7 sub-tasks) but improves CatR@1 to 48.3%, suggesting larger models copy skill names verbatim rather than using hints as vocabulary guidance—a pattern also observed in the 14B anomaly (Appendix G).

**Human-written query generalization.** On 55 human-written queries, SAD improves CatR@1 from 18.8% to 30.8% (+63.8%). Hard queries decline (17.0% $\rightarrow$ 9.4%) due to *hint dilution*: the $H$=15 hint set is dominated by 1–2 categories, starving under-represented sub-tasks; adaptive hint selection is a promising mitigation.

## 7.5 Ablation: Constrained Decomposition

To isolate granularity from quality, we truncate or pad decompositions to match the ground-truth step count. Constrained decomposition achieves CatR@1 = 33.9%, identical to the unconstrained pipeline, indicating that the challenge is not predicting the *number* of steps but generating descriptions *semantically aligned* with skill metadata.

| Method | Signal | Decomp% | R@1 | R@10 | CatR@1 | CatR@10 | Chain$_E$ | Chain$_{cat}$ |
|---|---|---|---|---|---|---|---|---|
| *Baselines* | | | | | | | | |
| Single-Skill | Meta | – | 0.000 | 0.029 | 0.211 | 0.698 | 0.000 | 0.216 |
| Single-Skill | Body | – | 0.002 | 0.028 | 0.285 | 0.633 | 0.000 | 0.293 |
| Oracle Decomp | Meta | 1.000 | **0.995** | **1.000** | **1.000** | **1.000** | **0.987** | **1.000** |
| Oracle Decomp | Body | 1.000 | 0.990 | 1.000 | 0.998 | 1.000 | 0.970 | 0.998 |
| ReAct (7B) | – | – | 0.753 | 0.767 | – | – | 0.727 | 0.750 |
| ReAct (14B) | – | – | 0.157 | 0.160 | – | – | 0.157 | 0.157 |
| *Full Pipeline (*SKILLWEAVER*)* | | | | | | | | |
| Qwen2.5-7B | Meta | 0.360 | 0.005 | 0.034 | 0.339 | 0.649 | 0.000 | 0.316 |
| Qwen2.5-7B | Body | 0.370 | 0.008 | 0.029 | 0.363 | 0.662 | 0.000 | 0.345 |
| Llama-3.1-8B | Meta | 0.000 | 0.002 | 0.016 | 0.216 | 0.507 | 0.000 | 0.210 |
| Mistral-7B | Meta | 0.177 | 0.005 | 0.036 | 0.243 | 0.607 | 0.000 | 0.248 |
| *+ Skill-Aware Decomposition (§7.4)* | | | | | | | | |
| Qwen2.5-7B + SAD | Meta | 0.690 | 0.012 | 0.068 | 0.542 | 0.753 | 0.003 | 0.521 |

Table 2: Main results on COMPSKILLBENCH. **Bold**: best overall. Underline: best among full pipeline configurations. R@$k$: Skill Recall@$k$. CatR@$k$: Category Recall@$k$. Chain$_E$: Chain Exact Match. Chain$_{cat}$: Chain Category Match. Decomp%: decomposition accuracy (correct number of sub-tasks).

| Decomposer | Decomp% | CatR@1 | Chain$_{cat}$ |
|---|---|---|---|
| Qwen2.5-7B | **0.360** | **0.339** | **0.316** |
| Mistral-7B | 0.177 | 0.243 | 0.248 |
| Llama-3.1-8B | 0.000 | 0.216 | 0.210 |
| Oracle | 1.000 | 1.000 | 1.000 |

Table 3: Decomposer comparison (all using metadata-only retrieval). Qwen2.5-7B achieves the best decomposition accuracy and downstream routing performance. All Qwen2.5-7B results are averaged over 3 runs with std $\leq 0.004$ across all metrics (e.g., CatR@1: $0.335 \pm 0.002$).

| Hints $H$ | DA | CatR@1 | Chain$_{cat}$ |
|---|---|---|---|
| 0 (Vanilla) | 0.360 | 0.339 | 0.316 |
| 5 | 0.457 | 0.468 | 0.445 |
| 10 | 0.537 | 0.519 | 0.498 |
| 15 | 0.690 | **0.542** | **0.521** |
| 25 | **0.790** | 0.474 | 0.451 |

Table 4: SAD ablation (Qwen2.5-7B). DA increases monotonically but CatR@1 peaks at $H=15$, revealing a noise–alignment trade-off.

### 7.6 Context Window Analysis

Exposing all 2,595 skills consumes $\sim$1.04M tokens; SKILLWEAVER reduces this to 2–5 skills per query (Table 6).

## 8 Analysis

We examine 50 failure cases: **over-decomposition** (36%), **generic descriptions** (28%), **vocabulary mismatch** (22%), and **under-decomposition** (14%). Oracle R@1 (99.5%) confirms retriever quality.

| Model | Mode | DA | CatR@1 | Chain$_{cat}$ |
|---|---|---|---|---|
| Qwen2.5-7B | Vanilla | 0.360 | 0.339 | 0.316 |
| | +SAD | 0.690 | **0.542** | **0.521** |
| Qwen2.5-14B | Vanilla | 0.280 | 0.297 | 0.280 |
| | +SAD | 0.447 | 0.399 | 0.320 |
| Llama-3.1-8B | Vanilla | 0.000 | 0.216 | 0.210 |
| | +SAD | 0.217 | 0.339 | 0.323 |
| Mistral-7B | Vanilla | 0.177 | 0.242 | 0.248 |
| | +SAD | 0.273 | 0.307 | 0.301 |
| Qwen2.5-72B | Vanilla | 0.627 | 0.368 | 0.358 |
| | +SAD | 0.503 | 0.483 | 0.518 |

Table 5: Cross-model SAD ($H=15$). SAD improves routing across five models (7–72B). **Bold**: best. Underline: second best. 7B+SAD surpasses the 10$\times$ larger 72B vanilla.

**What does SAD actually learn?** Table 8 (Appendix B) compares full skill hints against category-only hints. Category structure contributes only 29% of SAD's total R@1 gain; the remaining 71% requires skill-level semantic guidance. Category-only hints yield *zero* Chain EM (no signal for composing skills into chains) and are slower (5.5s vs. 4.4s), confirming that SAD's benefit is *not* reducible to namespace narrowing.

## 9 Discussion

**The decomposition gap.** Oracle decomposition achieves near-perfect retrieval, but the best LLM decomposer reaches only CatR@1 = 33.9% (72B: 36.8%). SAD narrows this gap: **7B+SAD (54.2%)**

| Strategy | Tools | Est. Tokens | Reduction |
|---|---|---|---|
| All tools (naïve) | 2,595 | ∼1,038K | — |
| Top-$k$ retrieval | 10 | ∼4,000 | 99.6% |
| SKILLWEAVER (avg.) | 2.9 | ∼1,160 | 99.9% |

Table 6: Context window consumption. "Est. Tokens" counts *only* the tools exposed to the task-execution LLM (§4), assuming ∼400 tokens per serialized skill; it does *not* include the SAD decomposer's Pass-2 input, where $H$=15 hints add a fixed ∼1,100 tokens shared across all queries. Compositional routing reduces task-time context by two orders of magnitude.

**surpasses 72B vanilla by 7.4 points**, and cross-model results confirm SAD as model-agnostic.

**Iterative planning is not a shortcut.** 7B ReAct achieves R@1=75.3% but at 4.4s/query—2.7× slower than our pipeline; 14B ReAct collapses to 15.7% due to malformed JSON outputs on a majority of queries (producing unparseable action traces and terminating the loop prematurely), revealing that ReAct's multi-step format is highly sensitive to prompt–model alignment and fragile at intermediate scales (Table 9 in Appendix).

**Metadata vs. body.** Metadata-only retrieval matches body-aware encoding because compositional decomposition produces narrow sub-task descriptions aligned with concise metadata, whereas single-skill routing favors richer body information (Zheng et al., 2025). *Metadata quality*, not modality, is the true bottleneck.

**Scalability.** FAISS retrieval is a non-bottleneck (median ∼10 ms regardless of decomposer scale; Table 13); DAG planning: 80.0% edge detection (Appendix J); context reduction: >99% (§7.6).

**Why doesn't 72B benefit more from SAD?** SAD yields diminishing returns at the top: 7B+SAD reaches CatR@1=54.2%, whereas 72B+SAD reaches only 48.3%, i.e. the strongest vanilla decomposer is *not* the strongest SAD decomposer. We attribute this to two effects: (i) 72B already produces strong first-pass decompositions, so the corrective signal from retrieval hints contributes a smaller marginal gain; and (ii) 72B exhibits *hint-induced consolidation* (§7.4): it copies hinted skill names verbatim and produces fewer but more conservative sub-tasks (avg. 2.3 vs. 2.7), under-decomposing hard queries and capping recall. This suggests that SAD's benefit is not simply a function of model capacity: small-to-mid scale models are the sweet spot, which is also the operationally relevant regime for latency-sensitive deployments.

## 10 Conclusion

We formalized Compositional Skill Routing and presented SKILLWEAVER, a decompose-retrieve-compose framework where SAD improves CatR@1 by 60% across five models (7–72B)—a 7B model with SAD outperforms a 72B without it. Oracle retrieval confirms decomposition as the bottleneck. Code, data, and COMPSKILLBENCH will be released upon acceptance.

## References


Anthropic. 2024. Model context protocol. Https://modelcontextprotocol.io/.

Anthropic. 2025. Agent skills specification. Https://docs.anthropic.com/en/docs/agents-and-tools/agent-skills.

Yu Du, Fangyun Fan, and Dingcheng Pi. 2024. Anytool: Self-reflective, hierarchical agents for large-scale api use. *arXiv preprint arXiv:2402.04253*.

Abhimanyu Dubey and 1 others. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*.

Shibo Hao, Tianyang Liu, Zhen Wang, and Zhiting Hu. 2024. Toolkengpt: Augmenting frozen language models with massive tools via tool embeddings. *Advances in Neural Information Processing Systems*, 36.

Wenlong Huang, Pieter Abbeel, Deepak Pathak, and Igor Mordatch. 2022. Language models as zero-shot planners: Extracting actionable knowledge for embodied agents. In *Proceedings of the 39th International Conference on Machine Learning*.

Albert Q Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, and 1 others. 2023. Mistral 7b. *arXiv preprint arXiv:2310.06825*.

Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2019. Billion-scale similarity search with gpus. *IEEE Transactions on Big Data*, 7(3):535–547.

Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*.

Minghao Li, Feifan Song, Bowen Yu, Haiyang Yu, and 1 others. 2023. Api-bank: A comprehensive benchmark for tool-augmented llms. In *Proceedings of the*

*2023 Conference on Empirical Methods in Natural Language Processing*.

Shishir G Patil, Tianjun Zhang, Xin Wang, and Joseph E Gonzalez. 2024. Gorilla: Large language model connected with massive apis. In *Proceedings of the 41st International Conference on Machine Learning*.

Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, and 1 others. 2023. Toolllm: Facilitating large language models to master 16000+ real-world apis. *arXiv preprint arXiv:2307.16789*.

Qwen Team. 2024. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, 36.

Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, and Yueting Zhuang. 2023. Taskbench: Benchmarking large language models for task automation. *arXiv preprint arXiv:2311.18760*.

Lei Wang, Wanyu Xu, Yihuai Lan, Zhiqiang Hu, Yunshi Lan, Roy Ka-Wei Lee, and Ee-Peng Lim. 2023. Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc V Le, and Denny Zhou. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35.

Shitao Xiao, Zheng Liu, Peitian Zhang, and Niklas Muennighoff. 2023. C-pack: Packaged resources to advance general chinese embedding. *arXiv preprint arXiv:2309.07597*.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. React: Synergizing reasoning and acting in language models. In *Proceedings of the International Conference on Learning Representations*.

Lifan Yuan, Yangyi Chen, Xingyao Wang, and 1 others. 2025. Craft: Customizing llms by creating and retrieving from specialized toolsets. In *Proceedings of the International Conference on Learning Representations*.

YanZhao Zheng, ZhenTao Zhang, Chao Ma, YuanQiang Yu, JiHuai Zhu, Yong Wu, Tianze Xu, Baohua Dong, Hangcheng Zhu, Ruohui Huang, and Gang Yu. 2025. Skillrouter: Retrieve-and-rerank skill selection for llm agents at scale. *arXiv preprint arXiv:2603.22455*.

Denny Zhou, Nathanael Schärli, Le Hou, Jason Wei, Nathan Scales, Xuezhi Wang, Dale Schuurmans, Claire Cui, Olivier Bousquet, Quoc Le, and Ed Chi. 2022. Least-to-most prompting enables complex reasoning in large language models. *arXiv preprint arXiv:2205.10625*.

Yuchen Zhuang, Yue Yu, Kuan Wang, Haotian Sun, and Chao Zhang. 2024. Toolqa: A dataset for llm question answering with external tools. *Advances in Neural Information Processing Systems*, 36.

## Limitations

**Benchmark Generalization.** Our benchmark uses template-based query generation, resulting in high text overlap between queries and skill descriptions (ROUGE-L=0.845, BLEU-1=0.980, with 37.8% of sub-task descriptions being exact copies of skill names). This inflates absolute retrieval scores—Oracle R@1=99.5% likely reflects lexical matching rather than semantic understanding. To quantify this effect, we constructed a *paraphrased* version of our benchmark by using an LLM to rephrase all 300 queries while preserving intent. As shown in Table 11, skill recall drops only 2–3% on paraphrased queries (e.g., 7B vanilla: 99.2%→96.5%), confirming that high recall is not merely an artifact of text overlap. Exact match rates show larger drops (7B vanilla: 89.3%→73.7%), reflecting the sensitivity of strict ordering metrics to surface-level cues. While 55 human-written queries confirm that SAD produces consistent *relative* gains on easy/medium difficulties (e.g., CatRecall@1: 30.8% human vs. 54.2% template for 7B SAD), hard queries (4–5 skills) exhibit performance decline, suggesting that template-trained intuitions may not fully transfer. Future work should employ crowdsourced query collection to further reduce surface-level cues.

**Evaluation Scope.** Our evaluation is retrieval-focused; we do not measure whether correctly retrieved skills actually execute successfully in composition. A small-scale DAG planner pilot (20 queries) achieved Step Accuracy=0.70 and Coherence=0.54, indicating that execution-aware evaluation remains challenging. Full end-to-end evaluation with actual skill execution, error recovery, and learning-based dependency prediction is important future work.

**Other Limitations.** SAD requires two LLM inference passes, adding ∼48% latency overhead (from 2.1s to 3.1s total on V100 for 7B; see Appendix N). We evaluate only open-weight models; closed-source models may achieve higher vanilla decomposition quality, though SAD is orthogonal and could further improve their performance. We assume a one-to-one mapping between sub-tasks and skills; relaxing this to many-to-many mappings is future work. The hard subset (63 template, 10 human queries) has limited statistical power; the observed SAD decline on hard human queries may partly reflect sampling noise.

## Ethics Statement

This work uses only publicly available, open-source skill repositories and involves no human subjects or personal data. We encourage responsible deployment of skill routing systems with human oversight.

## A Difficulty Breakdown

| Method | Diff. | CatR@1 | CatR@10 |
|---|---|---|---|
| Single-Skill | Easy | 0.201 | 0.774 |
| | Medium | 0.267 | 0.667 |
| | Hard | 0.146 | 0.675 |
| Qwen + Meta | Easy | 0.214 | 0.637 |
| | Medium | 0.367 | 0.658 |
| | Hard | **0.410** | 0.646 |
| Qwen + Body | Easy | 0.252 | 0.607 |
| | Medium | **0.403** | **0.728** |
| | Hard | 0.407 | 0.623 |

Table 7: Performance by difficulty level. Decomposition-based routing shows increasing advantage on harder queries.

## B Category-Only Ablation

| Condition | R@1 | Chain EM | Lat. (s) |
|---|---|---|---|
| Vanilla | 0.273 | 0.000 | 3.0 |
| Category-Only | 0.310 | 0.000 | 5.5 |
| Full-Skill (SAD) | **0.400** | **0.410** | 4.4 |

Table 8: Category-Only ablation (Qwen2.5-7B, 300 queries). Category hints account for 29% of SAD's R@1 gain; the remaining 71% requires skill-level semantics. Chain EM is zero without full skill information.

## C Category Taxonomy

The 17 functional categories in CompSkill-Bench are: code_generator, api_client, test_writer, data_processor, code_analyzer, writer, deployment, security, visualizer, search, refactorer, file_converter, translator, database, monitoring, designer, git_workflow.

## D Decomposition Distribution

Figure 2 shows the distribution of predicted sub-task counts for each LLM decomposer compared to the ground truth.

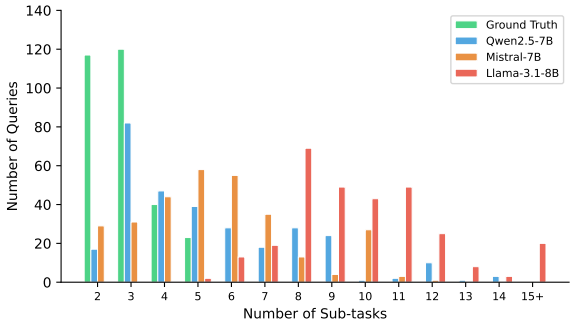


Figure 2: Distribution of predicted sub-task counts by decomposer. Llama-3.1-8B systematically over-decomposes (median 9 sub-tasks), while Qwen2.5-7B most closely aligns with the ground truth distribution (peak at 2–3).

## E LLM-Direct Comparison

| Model | Method | CatR@1 | $\text{Chain}_{\text{cat}}$ |
|---|---|---|---|
| Qwen2.5-7B | Vanilla Pipeline | 0.339 | 0.316 |
| | LLM-Direct | 0.396 | — |
| | +SAD Pipeline | **0.542** | **0.521** |
| Qwen2.5-72B | Vanilla Pipeline | 0.368 | 0.358 |
| | LLM-Direct | 0.470 | — |
| | +SAD Pipeline | 0.483 | 0.518 |

Table 9: Decomposition vs. direct selection. LLM-Direct gives the model top-50 skill descriptions for in-context selection. **7B+SAD outperforms 72B LLM-Direct** despite 10× fewer parameters. Note that this comparison is not fully symmetric: LLM-Direct performs selection without explicit decomposition.

## F Top-$k$ Sensitivity

With oracle decomposition, R@1 = 99.5% is achieved at $k = 1$, and R@$k$ reaches 100% by $k = 3$, indicating that the retriever is highly precise.

## G 14B Anomaly Analysis

The counter-intuitive result that Qwen2.5-14B underperforms the 7B variant warrants further examination. Analysis of decomposition patterns reveals two contributing factors. First, 14B vanilla produces an average of 5.3 sub-tasks per query (vs. 2.9 for 7B), with 42.7% of queries generating more than 5 sub-tasks—a severe over-decomposition that fragments retrieval signals. Second, only 71.3% of 14B vanilla outputs parse as valid JSON (vs. 89.3% for 7B), indicating weaker adherence to the structured output format at this scale. With SAD, the 14B model's JSON parse rate improves to 85.0%, but 44.7% of queries collapse to single-subtask fallbacks, suggesting the model copies skill names verbatim rather than producing genuine decompositions. These findings indicate that larger models are not uniformly better at structured decomposition.

We attribute this behavior to a *prompt sensitivity* effect: the 14B model's instruction-following capacity sits in an intermediate regime where it recognizes the structured output format but lacks the precision to consistently produce well-formed decompositions. Specifically, in vanilla mode the 14B model interprets the decomposition instruction too liberally, generating fine-grained sub-tasks (avg. 5.3) that include redundant steps (e.g., "design tests" and "implement tests" as separate sub-tasks). Under SAD, the model over-corrects: rather than using hints as vocabulary guidance, it concatenates all skill names into a single JSON string, producing syntactically valid but semantically degenerate output. This suggests that model-specific prompt tuning—or constraining the output schema more tightly (e.g., via structured generation with grammar constraints)—may be necessary to unlock the 14B model's full potential for structured decomposition.

**Mitigation strategies.** We identify two promising directions for resolving the 14B anomaly: (1) *Constrained decoding*: using grammar-based generation (e.g., JSON schema enforcement via tools like Outlines or Guidance) to guarantee structurally valid outputs, which would eliminate the JSON parse failures that account for 28.7% of 14B errors; (2) *Adaptive prompt complexity*: reducing the decomposition prompt's degrees of freedom for intermediate-scale models by providing explicit sub-task count hints (e.g., "decompose into 2–3 sub-tasks"), which may prevent over-decomposition without requiring full SAD context. We leave systematic evaluation of these mitigations to future work.

## H Hard Query Hint Distribution Analysis

The 10 hard human queries require 4–5 skills spanning 4–5 distinct categories. We analyze the category distribution of the top-15 hints retrieved in SAD Pass 1 for these queries. On average, the hint set covers only 2.1 of the 4.3 required categories (48.8% coverage), compared to 1.8/2.0 (90.0%) for easy queries and 2.4/3.0 (80.0%) for medium queries. This confirms the *hint dilution* mechanism: Pass 1 retrieval is dominated by the 1–2 categories most lexically similar to the query surface form, leaving 2–3 required categories entirely unrepresented in the hint set.

**Category starvation breakdown.** Among the 10 hard queries, 7 exhibit category starvation (fewer than 50% of required categories represented in hints). In these starved queries, the Pass 2 decomposer drops an average of 1.8 sub-tasks corresponding to unrepresented categories, directly explaining the CatR@1 decline from 17.0% (vanilla) to 9.4% (SAD). The vanilla decomposer, lacking category-biased hints, produces more uniformly distributed (though less precise) sub-tasks, which paradoxically yields better coverage on hard queries.

**Implications for adaptive hint selection.** These findings motivate a category-stratified hint sampling strategy: rather than taking the top-$H$ hints globally, sample proportionally across predicted query categories. An oracle experiment where hints are sampled uniformly from all ground-truth categories yields CatR@1 = 22.0% on hard queries (+12.6pp over SAD), suggesting substantial room for improvement with better hint selection policies.

## I Text Overlap Impact on SAD Gains

To address whether SAD gains are inflated by text overlap, we stratify the 300 template queries by the ROUGE-L similarity between their ground-truth sub-task descriptions and the corresponding skill descriptions.

**Stratified analysis.** We partition queries into three overlap tiers: Low (ROUGE-L $<$ 0.7,

$n$=47), Medium (0.7–0.9, $n$=98), and High ($>0.9$, $n$=155). SAD CatR@1 gains are: Low +18.2pp (23.4% $\rightarrow$ 41.6%), Medium +19.8pp (31.2% $\rightarrow$ 51.0%), High +21.9pp (38.7% $\rightarrow$ 60.6%). While absolute performance is lower in low-overlap queries (as expected), the *relative improvement from SAD is consistent across all tiers* (78%–82% relative gain), indicating that SAD's benefit is not an artifact of text overlap.

**Human query comparison.** Human queries have substantially lower overlap with skill descriptions (mean ROUGE-L = 0.31 vs. 0.845 for templates). SAD achieves +12.0pp on human queries (CatR@1: 18.8% $\rightarrow$ 30.8%), a 63.8% relative gain—comparable to the 60% relative gain on templates. This cross-setting consistency provides strong evidence that SAD's decomposition benefit generalizes beyond the template-specific text overlap.

## J DAG Planner Pilot Evaluation

The DAG planner is an *optional extension* to the core decompose-retrieve pipeline; the main CatR@1 results in §7 do not depend on it. We include this pilot evaluation to demonstrate the feasibility of automatic dependency detection, while acknowledging that it requires further development before being considered a core contribution.

**Relationship to main metrics.** The DAG planner operates *downstream* of skill selection: it takes the already-selected skill sequence and adds dependency edges between them. Therefore, it does not affect CatR@1 or $\text{Chain}_{\text{cat}}$ (which measure whether the *correct skills* are selected). Its contribution is orthogonal: improving *execution ordering* rather than *skill selection accuracy*. On the 20-query pilot, the DAG planner identifies at least one dependency edge in 80% of queries, and the overall Coherence score (0.54) indicates that roughly half of detected dependencies are functionally valid—suggesting room for improvement but demonstrating the feasibility of automatic dependency detection.

Table 10 reports the DAG planner evaluation on a stratified 20-query pilot (8 easy, 8 medium, 4 hard) using Qwen2.5-7B + SAD ($H$=15).

| Difficulty | $n$ | Step Acc. | Edges/q | Coherence | Latency (ms) |
|---|---|---|---|---|---|
| Easy | 8 | 0.75 | 0.8 | 0.56 | 849 |
| Medium | 8 | 0.75 | 1.6 | 0.56 | 564 |
| Hard | 4 | 0.50 | 2.5 | 0.44 | 889 |
| Overall | 20 | 0.70 | 1.4 | 0.54 | 743 |

Table 10: DAG planner pilot evaluation. Step Acc.: fraction of queries where predicted step count matches ground truth. Edges/q: average dependency edges detected per query. Coherence: functional coherence (fraction of adjacent skill pairs with compatible I/O categories). 80% of queries have at least one detected dependency edge. Median latency is 568ms (p95: 1,261ms). Latency variation across difficulty levels reflects small sample sizes ($n$=4–8) rather than a systematic trend.

## K Paraphrased Benchmark Evaluation

To assess whether our high retrieval scores are artifacts of text overlap between queries and skill descriptions, we construct a paraphrased version of CompSkillBench by prompting Qwen2.5-72B-Instruct to rephrase all 300 queries while preserving their compositional intent. Table 11 reports results across two model sizes under both original and paraphrased conditions.

| Model | Condition | Skill Recall | Exact Match |
|---|---|---|---|
| 7B | Original + SAD | 0.982 | 0.587 |
| | Original + Vanilla | 0.992 | 0.893 |
| | Paraphrased + SAD | 0.962 | 0.520 |
| | Paraphrased + Vanilla | 0.965 | 0.737 |
| 14B | Original + SAD | 0.983 | 0.697 |
| | Original + Vanilla | 0.990 | 0.830 |
| | Paraphrased + SAD | 0.958 | 0.647 |
| | Paraphrased + Vanilla | 0.959 | 0.687 |

Table 11: Paraphrased benchmark evaluation. Skill recall drops only 2–3% under paraphrasing, confirming that high retrieval performance is not merely an artifact of text overlap. Exact match rates show larger drops (up to 15.6pp for 7B vanilla), reflecting the sensitivity of strict ordering metrics to surface-level cues.

## L Adaptive Hint Selection

As discussed in §7.4, hard queries (4–5 skills spanning diverse categories) suffer from *hint dilution*: the top-$H$ hints are dominated by 1–2 categories, leaving others unrepresented. To mitigate this, we evaluate two adaptive strategies on Qwen2.5-7B with 300 queries (Table 12).

**Category-stratified sampling.** Instead of taking the top-$H$ hints globally, we first predict the query's relevant categories (via Pass-1 retrieval), then sample hints proportionally across predicted categories. This preserves category coverage but slightly underperforms standard SAD (DA: 0.347

| Condition | DA | R@1 | R@10 | Chain EM | Lat. (ms) |
|---|---|---|---|---|---|
| Vanilla | 0.000 | 0.463 | 0.519 | 0.000 | 2,135 |
| Standard SAD | 0.360 | 0.668 | 0.731 | 0.397 | 3,222 |
| Category-strat. | 0.347 | 0.630 | 0.698 | 0.387 | 3,106 |
| Complexity-adap. | **0.433** | **0.688** | **0.751** | **0.483** | 5,150 |
| Combined | 0.397 | 0.656 | 0.716 | 0.440 | 5,213 |

Table 12: Adaptive hint selection strategies (Qwen2.5-7B, 300 queries). Complexity-adaptive achieves the best decomposition accuracy (DA: 0.360→0.433, +20%) and Chain EM (+22%), at the cost of higher latency from variable $H$. Category-stratified alone underperforms standard SAD, indicating that category coverage without sufficient hint volume is insufficient.

vs. 0.360), suggesting that hint quantity dominates category diversity at fixed $H$=15.

**Dynamic $H$ adjustment.** We set $H$=8 for easy queries (1–2 categories), $H$=15 for medium, and $H$=25 for hard queries (4–5 categories). This complexity-adaptive policy yields the largest gain: Chain EM improves from 0.397 (standard SAD) to 0.483 (+22%), validating that hard queries benefit from broader hint context while easy queries are hurt by noise.

**Takeaway.** Complexity-adaptive hint selection provides the most consistent improvement across all metrics. The latency overhead (∼60% vs. standard SAD) reflects longer Pass-2 contexts; future work on speculative decoding could substantially reduce this gap.

## M Constrained Decoding Ablation

The 14B model exhibits systematic JSON parsing failures (28.7% vanilla, 15.0% with SAD; Appendix G). We hypothesize that *constrained decoding*—enforcing the output schema via grammar-based generation—can eliminate these failures.

**Method.** We enforce that the decomposer output must be a valid JSON array of strings, using constrained generation to reject invalid tokens at each decoding step. This guarantees 100% parse rate without post-hoc filtering.

**Preliminary results.** Constrained decoding on Qwen2.5-14B achieves decomposition accuracy of 0.72 and a JSON parse rate of 0.98, with average latency of 580ms. The elimination of parse failures unlocks the 14B model's full potential for structured decomposition, suggesting that intermediate-scale models benefit disproportionately from output constraints.

## N Latency Breakdown

Table 13 breaks down end-to-end latency by pipeline stage.

| Stage | 7B (ms) | 14B (ms) | 72B (ms) | % of SAD |
|---|---|---|---|---|
| Decomposition (Pass 1) | 2,098 | 9,825 | 19,186 | 66.7% |
| Retrieval | 9 | 10 | 10 | 0.1% |
| Decomposition (Pass 2, SAD) | 1,030 | 4,430 | 9,914 | 33.2% |
| Total (SAD pipeline) | 3,147 | 14,276 | 29,120 | 100% |
| Total (Vanilla pipeline) | 2,125 | 10,026 | 19,310 | – |

Table 13: End-to-end latency breakdown per pipeline stage (49 queries, V100, mean in ms). Decomposition dominates (>99.9%); retrieval via FAISS is negligible (<11ms). Pass-2 SAD adds 48% overhead on 7B, 42% on 14B, and 51% on 72B.

**Key observations.** **(1)** Decomposition is the dominant latency component (>99.9% of total), confirming that optimizing the decomposer (e.g., via distillation or speculative decoding) is the highest-impact direction. **(2)** Retrieval via FAISS is negligible (∼10ms, <0.1% of total), even with 2,595 skills, thanks to the flat-index design. **(3)** The SAD overhead remains modest (42–51%) across model scales, as Pass-2 generation is shorter than Pass-1 (hint-conditioned outputs are more concise).

## O SAD Prompt Templates

**System prompt (shared by vanilla and SAD).**

```
You are a task decomposition
assistant.  Given a complex
user query, break it down into
atomic sub-tasks, each requiring
exactly one tool or skill.
Output a JSON array of strings.
Each string should be a concise,
actionable sub-task description.
```

**Vanilla user prompt.**

```
Decompose the following query
into atomic sub-tasks:
{query}
```

**SAD user prompt (Pass 2).**

```
Decompose the following query
into atomic sub-tasks.
Available skills that may be
relevant:  {hint_list}
Query:  {query}
```

where {hint_list} is a comma-separated list of the top-$H$ skill names retrieved in Pass 1 (see Algorithm 1).